Trabajo Práctico Grupal

UTN FRBA - Algoritmos y Estructuras de Datos

Integrantes del Grupo:

Brian Nahuel Geronimo | Legajo: 235.089-0 | Email: bgeronimo@frba.utn.edu.ar

Milagros Abigail Lalli | Legajo: 233.979-1 | Email: mlalli@frba.utn.edu.ar

Andrea Ines Coria | Legajo: 233.792-7| Email: ancoria@frba.utn.edu.ar

Ezequiel Daniel MartÍnez | Legajo: 235.493-7 | Email: ezemartinez@frba.utn.edu.ar

Lucas Ezequiel Ricarte | Legajo: 233.015-5 | Email: lricarte@frba.utn.edu.ar

Carrera: Ingeniería en Sistemas de Información

Comisión: K1103

Ciclo Lectivo: 2026

Índice

1. Descripción de la solución e Hipótesis **2**
2. Diagrama de bloques de subprogramas **3**
3. División de tareas en el equipo **5**
4. Casos de prueba **6**
10

1. Descripción de la solución e Hipótesis

Estructuras y Memoria:

- **Matriz Espacial Estática:** Para simular el espacio aéreo sin GPS, se modeló el mapa utilizando una matriz bidimensional estática de 200x200 celdas. Esta estructura permite representar de forma unívoca cada coordenada espacial asociándola directamente a los índices del arreglo, lo que optimiza la búsqueda de rutas a una complejidad de tiempo constante (O(1)).
- **Separación de Estructuras (RAM vs Disco):** Se decidió utilizar una arquitectura de dos estructuras de datos independientes para optimizar el consumo. El struct OrdenArchivo se utiliza para interactuar con los archivos binarios (.dat), conteniendo las coordenadas X e Y de forma explícita. Por otro lado, el struct Orden reside puramente en la memoria RAM y su ubicación (X,Y) está implícita por su posición en la matriz, ahorrando espacio en memoria.
- **Centinela de fin de ruta y celdas nulas:** Dado que la coordenada (0, 0) representa un casillero válido dentro del mapa, se estableció el valor centinela -1 en los campos siguientex e siguientey para indicar de manera inequívoca el final de una trayectoria o señalar que un casillero no apunta a ningún otro lado.
- **Validación de Límites Espaciales:** Para prevenir errores de desbordamiento (segmentation fault) y corrupción de datos, se estableció como regla que cualquier acceso de lectura o escritura a la matriz debe ser filtrado previamente por la función de control estaEnRango, asegurando que las coordenadas solicitadas se mantengan estrictamente dentro del intervalo válido [0, 199].

2. Diagrama de bloques de subprogramas

Nota: Debido a la extensión de los diagramas de flujo, las representaciones gráficas de cada módulo han sido adjuntadas a tamaño completo al final de este documento en la sección de Anexo para evitar cortes de formato y garantizar su correcta legibilidad.

Operación 1 (cargarArchivoEnMemoria)

- Inicio de la función cargarArchivoEnMemoria().
- Apertura del archivo binario en modo lectura (rb). Si no se logra abrir, se emite mensaje de error y finaliza la rutina.
- Lectura secuencial (fread) volcando los datos a un arreglo temporal (registrosTemp).
- Validación inicial: El archivo debe tener al menos 2 registros. Si no cumple, aborta.
- Recorrido del arreglo temporal para validar reglas de negocio: límites de grilla mediante estaEnRango(), rechazo de coordenadas duplicadas, validación de un único despegue, validación de un único fin (aterrizaje o kamikaze), y exclusión mutua de acciones simultáneas (despegue sin ataques).
- Simulación del recorrido de la ruta: validación de saltos aledaños lógicos entre el registro

actual y sus coordenadas siguientex e siguientey, detección de huecos, y prevención de ciclos (loops).
- Si las validaciones son exitosas, se aplica inicializarMemoria() para limpiar la matriz.
- Volcado final de los datos del arreglo temporal a la matriz global memoria[200][200] estableciendo celdaOcupada = true. Emisión de confirmación exitosa.

Operación 2 (mostrarAtaqueCargado)

- Inicio de la función mostrarAtaqueCargado().
- Escaneo de la matriz memoria[200][200] identificando casilleros con celdaOcupada == true.
- Si la cantidad de registros encontrados es 0, se imprime "No hay ningun ataque cargado en memoria" y finaliza la rutina.
- Si existen registros, se calcula el total de páginas: totalPaginas = (cantidad + 15 - 1) / 15.
- Ciclo de despliegue: se imprime la cabecera, número de página, total de registros y la tabla tabulada con los 15 registros de la página activa.
- Lectura de comando de usuario: 'N' incrementa el índice de página si no es la última; 'A' decrementa si no es la primera; 'S' finaliza el ciclo de navegación.

Operación 3 (CrearArchivo)

- Inicio de la función CrearArchivo().
- Apertura o creación del archivo en modo escritura binaria (wb). Si falla, se notifica y finaliza.
- Inicio de ciclo de carga interactiva controlado por confirmación del usuario ('s').
- Solicitud de ingreso de datos por teclado para conformar el struct: Coordenadas X e Y, tiempo de espera, banderas lógicas de ataque/despegue/aterrizaje y coordenadas del salto siguiente.
- Escritura del registro instanciado directamente en el archivo binario mediante fwrite().
- Consulta al usuario para continuar agregando órdenes. Si responde 'n', se cierra el archivo (fclose) y se imprime mensaje de éxito.

Operación 4 (EditarArchivo)

- Inicio de la función EditarArchivo().
- Apertura del archivo en modo lectura y escritura binaria (r+b). Si falla, aborta.
- Solicitud de las coordenadas X e Y correspondientes al registro a editar.
- Búsqueda secuencial en disco (fread) iterando registro a registro e incrementando un contador de posición, hasta encontrar coincidencia absoluta con X e Y.
- Si se agota la búsqueda sin hallazgos, se emite error y finaliza.
- Al encontrarlo, impresión en pantalla de los datos actuales y solicitud de confirmación de edición.
- Si el usuario confirma, ingreso de los nuevos valores lógicos por teclado.
- Reubicación exacta del puntero en el archivo mediante fseek(f2, posicion * sizeof(OrdenArchivo), SEEK_SET).
- Sobrescritura directa del registro mediante fwrite() y cierre del archivo.

Operación 5 (corregirRegistroMemoria)

- Inicio de la función corregirRegistroMemoria().
- Solicitud de coordenadas X e Y al usuario por teclado.

- Validación de límites: si !estaEnRango(x, y), se emite mensaje de error y finaliza.
- Validación de ocupación: si la celda no contiene orden previa, se emite mensaje de error y finaliza.
- Impresión detallada del registro actual y confirmación de edición (s/n). Si se rechaza, finaliza.
- Ingreso de nuevos valores para espera, granadas, kamikaze, aterrizaje, despegue y salto siguiente.
- Validación integral de reglas lógicas del dron. Si alguna falla, se cancela la escritura.
- Aplicación de cambios en memoria[x][y] y emisión de confirmación "Registro modificado correctamente".

Operación 6 (GuardarMemoriaEnArchivo)

- Inicio de la función GuardarMemoriaEnArchivo().
- Creación del archivo binario de destino en modo escritura (wb).
- Recorrido matricial anidado iterando sobre filas y columnas.
- Condición de volcado: si celdaOcupada == true, se instancia un struct temporal OrdenArchivo, transcribiendo los campos lógicos residentes en RAM y añadiendo explícitamente sus coordenadas X e Y.
- Escritura del registro en el disco mediante fwrite().
- Fin del recorrido matricial, cierre del archivo e impresión de la cantidad total de registros exportados.

Operación 7 (visualizarEnHTML)

- Inicio de la función visualizarEnHTML().
- Apertura de un archivo de texto en modo escritura (w).
- Inyección de cabeceras estructurales de la web y estilos CSS (<head>, <style>, <table>) mediante sentencias fprintf.
- Recorrido secuencial anidado sobre las 200 filas y 200 columnas de la matriz.
- Condicional de renderizado: si la celda está ocupada, se evalúan las banderas para asignar el formato visual correspondiente (fondo amarillo 'D' para despegue; fondo rojo 'F' para finalización; fondo verde 'X' para pasos intermedios).
- Si la celda carece de actividad, se inyecta una etiqueta de celda vacía <td></td>.
- Cierre iterativo de filas y clausura final del cuerpo HTML (</table></body></html>).

3. División de tareas en el equipo

Lucas Ezequiel Ricarte (Estructuras y Memoria base): Modelado de las estructuras fundamentales del sistema (Orden para la RAM y OrdenArchivo para disco). Definición de las constantes de dimensión, declaración de la matriz espacial estática de 200x200 celdas y desarrollo de las funciones core de inicialización de memoria y validación de límites espaciales (estaEnRango).

Milagros Abigail Lalli (Operación 1 - Carga y Validaciones lógicas): Desarrollo del algoritmo de lectura segura de archivos binarios y volcado a arreglos temporales. Programación del motor de validación estricta de rutas: detección de saltos no aledaños mediante cálculo de diferencias absolutas, prevención de ciclos (loops), verificación de despegue/aterrizaje únicos y detección de celdas aisladas

o huecos.

Andrea Ines Coria (Operaciones 2 y 5 - Visualización y Edición en RAM): Desarrollo de la interfaz de salida estructurada por consola. Implementación de un sistema de paginación interactiva (bloques de 15 registros) para evitar el desbordamiento de pantalla al mostrar el ataque cargado. Además, desarrolló la lógica de modificación segura de registros directamente en memoria, validando las reglas de negocio (incompatibilidad de despegue y ataques simultáneos) antes de aplicar los cambios.

Brian Nahuel Geronimo (Operaciones 3 y 4 - Creación y Edición en Disco): Desarrollo del módulo de persistencia y manipulación directa de archivos .dat. Programación del asistente interactivo para instanciar misiones de ataque nuevas desde cero (Op. 3) y la lógica de posicionamiento de punteros (fseek) para buscar, leer y sobrescribir registros defectuosos de forma quirúrgica directamente en el archivo binario sin tener que volcarlo a la RAM (Op. 4).

Ezequiel Daniel Martínez (Menú Principal, HTML y Volcado a disco): Desarrollo del archivo main.cpp implementando el bucle interactivo y orquestando la comunicación entre todos los módulos del equipo. Además, se encargó de la extracción de datos programando la Operación 6 (recorrido de la matriz para generar archivos de *backup*) y la Operación 7 (renderización gráfica mediante inyección de etiquetas HTML dinámicas usando fprintf para visualizar la grilla en el navegador web).

4. Casos de prueba

Operación 1

Caso de Prueba 1: Trayectoria Válida Completa

Objetivo: Validar que un archivo binario íntegro sea procesado, validado y transferido exitosamente a la matriz global.

Procedimiento: Se ejecuta el programa y se selecciona la opción 1, se ingresa un archivo de test válido: Despegue en (0,0) con salto a (0,1); soltarGranada1 en (0,1) con salto a (1,1); Aterrizaje en (1,1) con coordenadas siguientes POS_NULA.

Resultado: El sistema valida la integridad del archivo sin advertencias, vuelca la información en memoria[x][y] y emite en consola: "*Archivo cargado con éxito en memoria*", "*El archivo fue validado y cargado con éxito*".

```
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en html

1
---CARGAR ARCHIVO DE ATAQUE---
Ingrese la ruta y nombre del archivo binario
test_valido.dat
Archivo cargado con éxito en memoria
El archivo fue validado y cargado con éxito
```

Caso de Prueba 2: Rechazo por Despegue Armado

Objetivo: Verificar la detección y el bloqueo de archivos que lancen ataques en la fase de despegue.

Procedimiento: Se suministra un archivo de test con error: el casillero inicial posee despegue = true y soltarGranada1 = true simultáneamente.

Resultado: Se detecta la incompatibilidad y se aborta inmediatamente bajo la leyenda: "*ERROR: no se pudo cargar el archivo. El archivo no cumple con las restricciones o no existe*".

```
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en html

1
---CARGAR ARCHIVO DE ATAQUE---
Ingrese la ruta y nombre del archivo binario
test_error_despegue_armado.dat
ERROR: La instrucción de despegue no puede estar acompañada por ningún ataque.
```

Caso de Prueba 3: Detección de Salto no Aledaño

Objetivo: Comprobar la efectividad de celdasAled al detectar saltos discontinuos entre casilleros distantes.

Procedimiento: Se intenta cargar un test con error: orden inicial en (0,0) cuyo destino apunta directamente a (5,5).

Resultado: Durante la simulación de ruta, la función intercepta el desplazamiento y cancela la operación emitiendo en consola: "*ERROR: Salto no aledaño desde (0,0) hacia (5,5)*".

```
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en html

1
---CARGAR ARCHIVO DE ATAQUE---
Ingrese la ruta y nombre del archivo binario
test_error_no_aledano.dat
ERROR: Salto no aledaño desde (0, 0) hacia (5, 5).
```

Caso de Prueba 4: Detección de Ruta Incompleta

Objetivo: Constatar que el sistema identifique las bifurcaciones rotas o referencias hacia casillas que no han sido declaradas.

Procedimiento: Se intenta cargar un archivo de test con error: Despegue en (0,0), salto a (0,1) cuyo registro de orden es omitido; aterrizaje en (1,1).

Resultado: La función buscarXCoordenadas() retorna -1 al rastrear el destino del despegue y se emite en consola: "*ERROR: Ruta incompleta. Hueco hacia (0,1)*".

```
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en html

1
---CARGAR ARCHIVO DE ATAQUE---
Ingrese la ruta y nombre del archivo binario
test_error_hueco.dat
ERROR: Ruta incompleta. Hueco hacia (0, 1).
```

Operación 2: (mostrarAtaqueCargado)

●Inicio de la función mostrarAtaqueCargado().

●Escaneo de la matriz memoria[200][200] identificando casilleros con celdaOcupada == true.

●Si la cantidad de registros encontrados es 0, se imprime "No hay ningun ataque cargado en memoria" y finaliza la rutina.

●Si existen registros, se calcula el total de páginas: totalPaginas = (cantidad + 15 - 1) / 15.

●Ciclo de despliegue: se imprime la cabecera, número de página, total de registros y la tabla tabulada con los 15 registros de la página activa.

●Lectura de comando de usuario: 'N' incrementa el índice de página si no es la última; 'A' decrementa si no es la primera; 'S' finaliza el ciclo de navegación.

Operaciones 3 y 4:

Bloque A: CrearArchivo

- Inicio de CrearArchivo(nombreArc).
- Apertura del archivo en modo "wb" y comprobación de que se haya creado correctamente.
- Solicitud de los datos de una orden: coordenadas X e Y, espera, acciones y coordenadas siguientes.
- Escritura del registro completo mediante fwrite(&orden, sizeof(OrdenArchivo), 1, f1).
- Consulta al usuario para determinar si desea ingresar otra orden.
- Repetición de la carga mientras la respuesta sea 's' o 'S'.
- Cierre del archivo con fclose e impresión del mensaje de confirmación.

Bloque B: EditarArchivo

- Inicio de EditarArchivo(nombreArc) y apertura del archivo en modo "r+b".
- Ingreso de las coordenadas X e Y que identifican el registro a modificar.
- Lectura secuencial con fread y comparación de las coordenadas de cada registro.
- Si no se encuentra coincidencia, se informa el error, se cierra el archivo y finaliza la función.
- Si se encuentra, se muestran todos los datos del registro y se solicita confirmación.
- Si el usuario cancela, se cierra el archivo sin modificar el registro.
- Si confirma, se solicitan los nuevos valores de los campos editables.
- Uso de fseek para volver al comienzo del registro y de fwrite para sobrescribirlo.
- Cierre del archivo y mensaje de modificación correcta.

Bloque C: GuardarMemoriaEnArchivo

- Inicio de GuardarMemoriaEnArchivo(nombreArc, memoria).
- Creación del archivo de destino en modo "wb" y comprobación de apertura.
- Recorrido completo de la matriz mediante los índices X e Y.
- Evaluación de memoria[x][y].celdaOcupada para cada posición.
- Si la celda está ocupada, se copian sus datos a una variable OrdenArchivo y se escribe el registro con fwrite.
- Incremento del contador registrosGuardados por cada registro escrito.
- Cierre del archivo e impresión de la cantidad total de registros guardados.

4. Casos de Prueba y Evidencias de Ejecución

Caso de Prueba 1: Creación de un archivo con varias órdenes

Objetivo: comprobar que CrearArchivo permita almacenar más de un registro binario.

Procedimiento: ejecutar la función, ingresar los datos de una primera orden, responder 's' para agregar otra y luego responder 'n'.

```
Seleccione una opcion: 3

--- CREAR ARCHIVO DE ATAQUE NUEVO ---
Ingrese la ruta y nombre del archivo a crear: posicion1

--- Ingresar Datos de la Orden ---
Coordenada X: 01
Coordenada Y: 03
Espera: 14
Soltar granada 1? (0=no, 1=si): 0
Soltar granada 2? (0=no, 1=si): 0
Ataque Kamikaze? (0=no, 1=si): 0
Aterrizaje? (0=no, 1=si): 0
Despegue? (0=no, 1=si): 1
Siguiente coordenada X (-1 si es fin): 01
Siguiente coordenada Y (-1 si es fin): 04

Deseas agregar otra orden? (s/n): s

--- Ingresar Datos de la Orden ---
Coordenada X: 01
Coordenada Y: 04
Espera: 12
Soltar granada 1? (0=no, 1=si): 1
Soltar granada 2? (0=no, 1=si): 0
Ataque Kamikaze? (0=no, 1=si): 0
Aterrizaje? (0=no, 1=si): 0
Despegue? (0=no, 1=si): 0
Siguiente coordenada X (-1 si es fin): 01
Siguiente coordenada Y (-1 si es fin): 05
```

Resultado: el archivo se crea y contiene los registros ingresados en el mismo orden. La función informa que el archivo fue creado y guardado correctamente.

Caso de Prueba 2: Búsqueda de un registro existente
Objetivo: verificar que EditarArchivo encuentre una orden a partir de sus coordenadas.

Procedimiento: abrir para edición un archivo que contenga al menos una orden e ingresar las coordenadas X e Y de uno de sus registros.

Resultado: se muestra el registro encontrado con sus campos actuales y se solicita confirmación para modificarlo.

Caso de Prueba 3: Búsqueda de coordenadas inexistentes
Objetivo: comprobar el comportamiento cuando no existe un registro con las coordenadas indicadas.

Procedimiento: ejecutar EditarArchivo e ingresar un par de coordenadas que no esté presente en el archivo.

Resultado: se muestra el mensaje de error indicando que no se encontró ningún registro, se cierra el archivo y no se realiza ninguna escritura.

```
Seleccione una opcion: 4

--- CORREGIR REGISTRO DEL ARCHIVO ---
Ingrese la ruta y nombre del archivo a editar: posicion1
Elija las coordenadas a editar en el archivo:
Coordenada X: 04
Coordenada Y: 17
ERROR: No se encontro ningun registro con esas coordenadas en el archivo.
```

Caso de Prueba 4: Cancelación de una modificación
Objetivo: verificar que el usuario pueda cancelar la edición sin alterar los datos almacenados.

Procedimiento: localizar un registro válido y responder 'n' cuando se consulta si desea modificarlo.

Resultado esperado: se informa que la modificación fue cancelada y el archivo permanece sin cambios.

```
Seleccione una opcion: 4

--- CORREGIR REGISTRO DEL ARCHIVO ---
Ingrese la ruta y nombre del archivo a editar: posicion1
Elija las coordenadas a editar en el archivo:
Coordenada X: 01
Coordenada Y: 04

========== REGISTRO ENCONTRADO ==========
X: 1 | Y: 4
Espera: 12
Granada 1: 1 | Granada 2: 0
Kamikaze: 0 | Aterrizaje: 0 | Despegue: 0
Siguiente X: 1 | Siguiente Y: 5
=========================================
Desea modificar este registro? (s/n): n
Modificacion cancelada.
```

Caso de Prueba 5: Modificación exitosa de un registro
Objetivo: comprobar que los nuevos valores sobrescriban el registro correcto sin alterar los demás.

Procedimiento: localizar una orden existente, confirmar con 's', ingresar nuevos valores y finalizar la operación.

Resultado: fseek posiciona el cursor en el registro identificado y fwrite escribe los datos actualizados. Se informa que el registro fue modificado correctamente.

```
Seleccione una opcion: 4

--- CORREGIR REGISTRO DEL ARCHIVO ---
Ingrese la ruta y nombre del archivo a editar: posicion1
Elija las coordenadas a editar en el archivo:
Coordenada X: 01
Coordenada Y: 04

========== REGISTRO ENCONTRADO ==========
X: 1 | Y: 4
Espera: 12
Granada 1: 1 | Granada 2: 0
Kamikaze: 0 | Aterrizaje: 0 | Despegue: 0
Siguiente X: 1 | Siguiente Y: 5
=========================================
Desea modificar este registro? (s/n): s

Ingrese los nuevos valores:
Nueva Espera: 42
Soltar Granada 1? (0/1): 0
Soltar Granada 2? (0/1): 1
Ataque Kamikaze? (0/1): 0
Aterrizaje? (0/1): 0
Despegue? (0/1): 0
Siguiente X: 02
Siguiente Y: 04

Registro en archivo modificado correctamente.
```

Operación 5: (corregirRegistroMemoria)

- Inicio de la función corregirRegistroMemoria().
- Solicitud de coordenadas X e Y al usuario por teclado.
- Validación de límites: si !estaEnRango(x, y), se emite mensaje de error y finaliza.
- Validación de ocupación: si la celda no contiene orden previa, se emite mensaje de error y finaliza.
- Impresión detallada del registro actual y confirmación de edición (s/n). Si se rechaza, finaliza.
- Ingreso de nuevos valores para espera, granadas, kamikaze, aterrizaje, despegue y salto siguiente.
- Validación integral de reglas lógicas del dron. Si alguna falla, se cancela la escritura.
- Aplicación de cambios en memoria[x][y] y emisión de confirmación "Registro modificado correctamente".

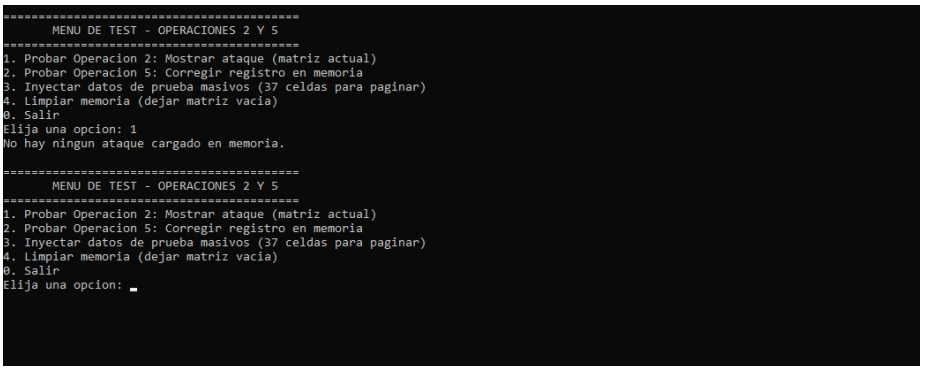
4. Casos de Prueba y Evidencias de Ejecución

Caso de Prueba 1: Manejo de matriz vacía (Operación 2)

- Objetivo: Verificar que el sistema no intente leer registros inexistentes ni falle al consultar una memoria recién inicializada.

- Procedimiento: Con la memoria en estado cero, se ingresa la opción 1 del menú de testeo (mostrarAtaqueCargado).

- Resultado: El software detecta la ausencia de órdenes activas y devuelve el aviso "No hay ningun ataque cargado en memoria.", regresando al menú principal de forma segura.

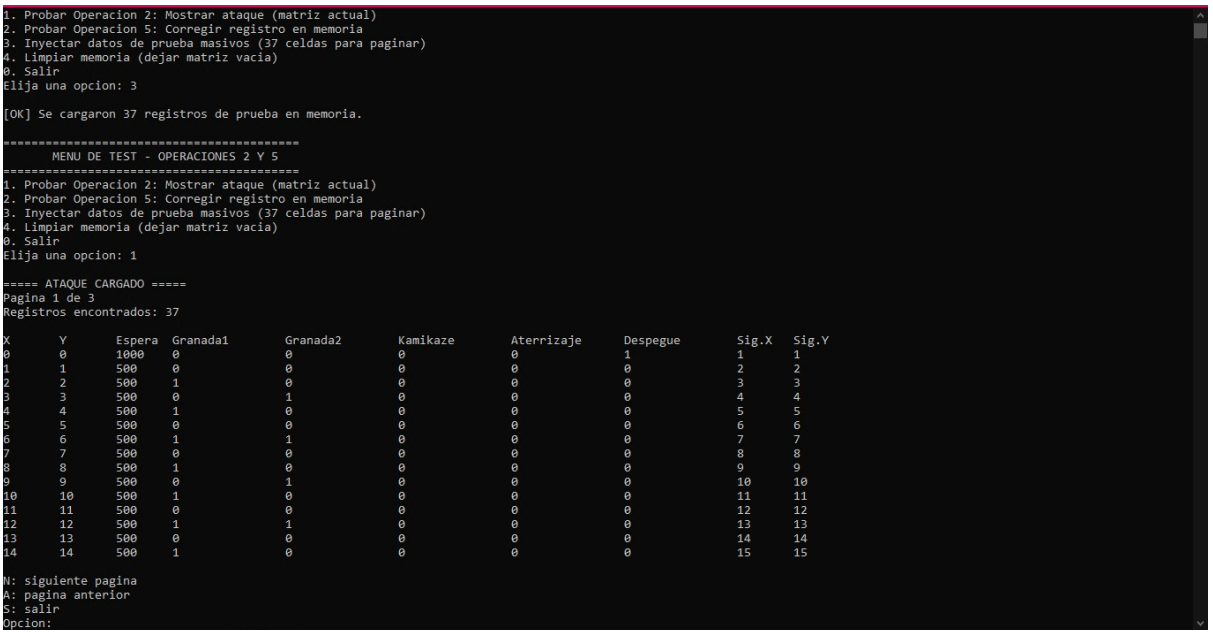


Caso de Prueba 2: Paginación y control de terminal (Operación 2)

- Objetivo: Constatar que ante un volumen masivo de datos no se desborde la pantalla y se respete la paginación interactiva requerida por la consigna.

- Procedimiento: Se cargan 37 registros de prueba y se ejecuta la visualización.

- Resultado: Se presentan los primeros 15 casilleros en formato tabular bajo el rótulo "Pagina 1 de 3" y el conteo "Registros encontrados: 37", ofreciendo los comandos interactivos para navegar entre páginas sin saturar la consola.



Caso de Prueba 3: Detección de accesos fuera de rango y celdas vacías (Operación 5)

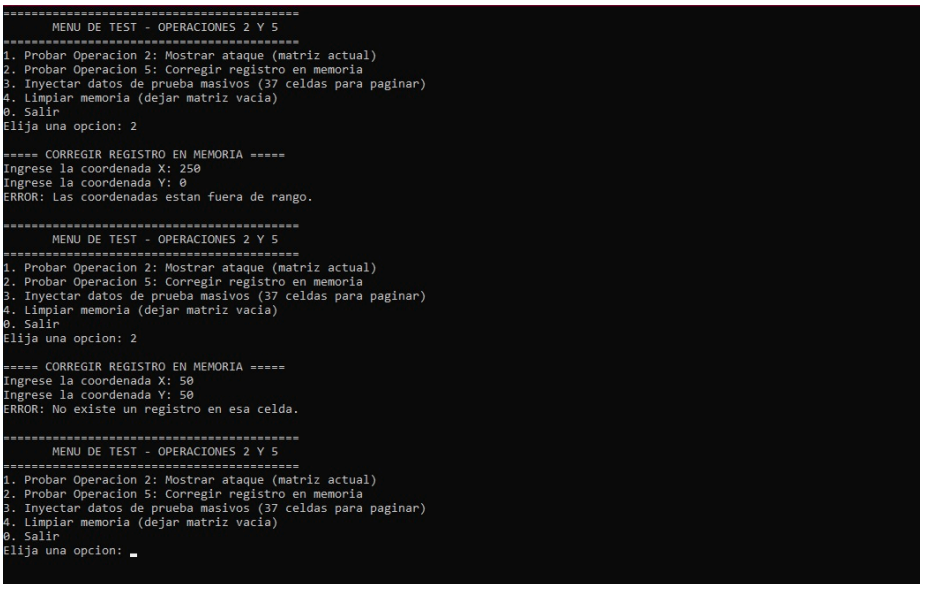
●Objetivo: Comprobar la protección de memoria contra índices inválidos y modificaciones de casilleros sin datos previos.

●Procedimiento:

none 1. Se solicita corregir la coordenada X = 250, Y = 0 (fuera de la grilla de 200x200).

none 2. Se solicita corregir la coordenada X = 50, Y = 50 (coordenada válida pero desocupada).

●Resultado: La rutina rechaza el primer intento con "ERROR: Las coordenadas estan fuera de rango." y el segundo con "ERROR: No existe un registro en esa celda.", impidiendo escrituras anómalas.

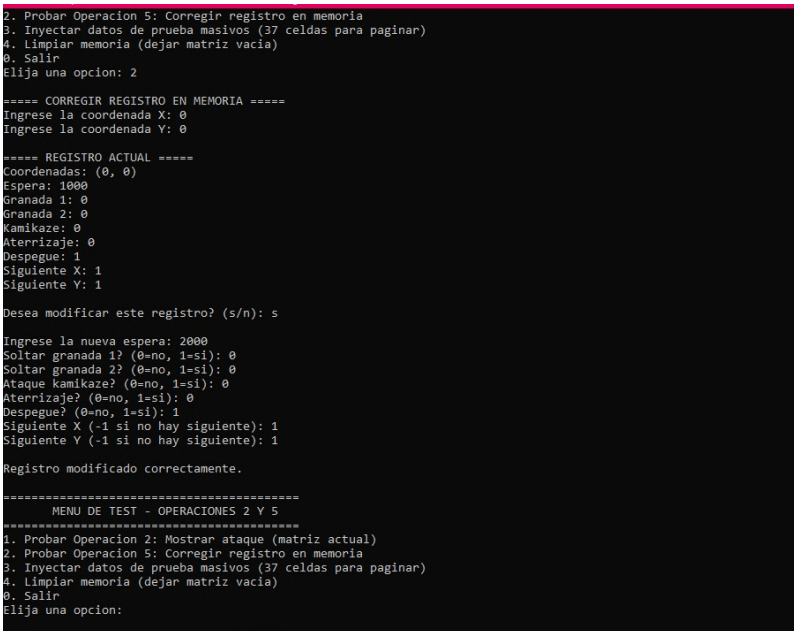


Caso de Prueba 4: Modificación interactiva exitosa en memoria (Operación 5)

●Objetivo: Validar la alteración directa y consistente de los datos de un casillero ocupado.

●Procedimiento: Se apunta a la celda (0,0), se visualiza el registro previo, se confirma la edición con 's' y se asignan nuevos parámetros válidos (Espera: 2000 ms, Despegue: 1, Salto a (1,1)).

●Resultado: Se aplican las modificaciones en el casillero correspondiente y el sistema confirma la operación con "Registro modificado correctamente.".



Caso de Prueba 5: Aserciones automatizadas de integridad (memoria.cpp)

●Objetivo: Demostrar por medio de tests unitarios formales (assert) la robustez de las funciones de memoria bajo condiciones de frontera.

●Procedimiento: Ejecución de batería automática que valida matriz vacía inicializada, direccionamiento en esquinas extremas (0,0) y (199,199), no invasión o desborde de celdas adyacentes (0,1) y (1,0) tras operaciones de escritura, y recorrido encadenado del dron entre celdas consecutivas hasta el casillero final.

●Resultado: Todas las aserciones del programa concluyen exitosamente, arrojando el mensaje "TODAS LAS PRUEBAS PASARON CORRECTAMENTE" y finalizando con código de retorno Process returned 0 (0x0).

Operación 6

Caso de Prueba 1: Guardado de la memoria en un archivo nuevo y verificación de consistencia
Objetivo: Comprobar que la función guardarMemoriaEnArchivo vuelca correctamente en un archivo binario los registros ocupados de la memoria activa y que el archivo resultante cumple con la especificación estructural OrdenArchivo, permitiendo su recarga íntegra sin errores.

Procedimiento:

Se ejecuta la Opción 1 para poblar la matriz en memoria a partir de un archivo binario existente (registro.dat).

Se selecciona la Opción 6 e ingresa como destino el nombre backup.dat.

Tras la confirmación del volcado, se vuelve a ejecutar la Opción 1 ingresando la ruta de backup.dat para comprobar su compatibilidad de lectura.

Resultado obtenido:

El programa crea backup.dat e informa exitosamente que "Se guardaron 3 registros en total", coincidiendo exactamente con la cantidad de órdenes de la misión inicial. Posteriormente, la Operación 1 procesa y carga backup.dat de forma limpia emitiendo el mensaje "Archivo cargado con éxito en memoria", demostrando que la serialización preservó intacta la estructura y el formato binario requerido.

```
Archivo creado y guardado con exito.


================================================
          SISTEMA DE GUIA DE APOYO - DRON
================================================
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en html
8. Salir del programa
================================================
Seleccione una opcion: 1

--- CARGAR ARCHIVO DE ATAQUE ---
Ingrese la ruta y nombre del archivo binario (ej: ataque.dat): registro.dat
Archivo cargado con éxito en memoria

================================================
          SISTEMA DE GUIA DE APOYO - DRON
================================================
1. Cargar archivo de ataque en memoria
2. Mostrar ataque cargado
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo
5. Corregir un registro en memoria
6. Guardar memoria en un archivo nuevo
7. Visualizar un archivo de ataque en html
8. Salir del programa
================================================
Seleccione una opcion: 6

--- GUARDAR MEMORIA A DISCO ---
Ingrese nombre del nuevo archivo binario (ej: backup.dat): backup.dat

Archivo 'backup.dat' creado con exito.
Se guardaron 3 registros en total.
```

Se buscamos comprobar que GuardarMemoriaEnArchivo guarda en un archivo binario los registros ocupados de la memoria y que el archivo generado pueda volver a cargarse, para esto cargamos registro.dat con la opción 1, elegimos la opción 6 e ingresamos un nuevo archivo de nombre “backup.dat” como nombre para un nuevo archivo que almacene los datos. Luego cargamos backup.dat con la opción 1.

Luego el programa da el resultado esperado, crea backup.dat e informa que se guardaron 3 registros, los mismos del archivo original. Al cargar backup.dat, el sistema lo acepta sin errores, por lo que el archivo conserva el formato OrdenArchivo.

Uso de Inteligencia Artificial durante el desarrollo del trabajo práctico

Durante el desarrollo del trabajo práctico se utilizó ChatGPT como herramienta de consulta y apoyo en distintas etapas de programación, integración y documentación. Su utilización permitió resolver dudas puntuales, analizar errores y evaluar posibles mejoras en la implementación.

Entre las principales instancias en las que se recurrió a esta herramienta se destacan:

- Diseño de estructuras y organización de la memoria: se consultó sobre la conveniencia de separar las estructuras utilizadas en memoria RAM y en archivos, considerando que las coordenadas de una matriz bidimensional pueden representarse mediante sus propios índices.

- Resolución de errores de compilación e integración: se utilizó asistencia para interpretar mensajes de error relacionados con conversiones de tipos de datos entre estructuras, ajustar parámetros de funciones y facilitar la comunicación entre los distintos módulos del programa.

- Optimización de validaciones: se consultaron alternativas para mejorar la búsqueda de coordenadas y la detección de registros duplicados, evitando recorridos innecesarios de la matriz y optimizando el proceso de carga.

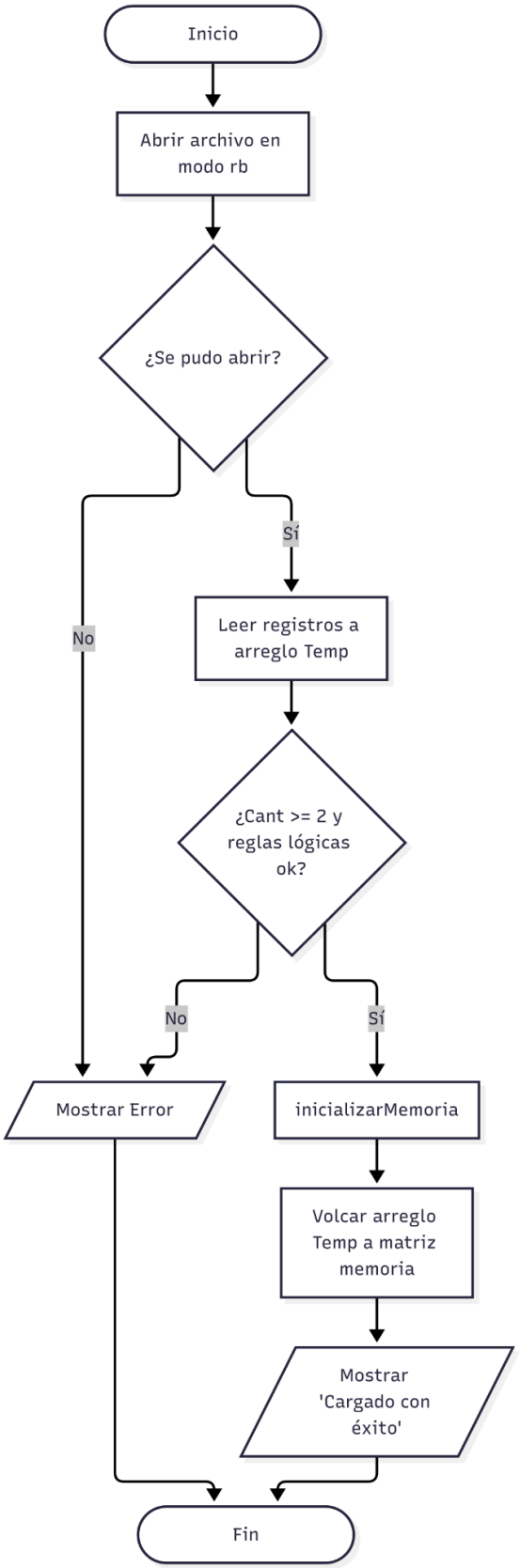
- Visualización y modificación de la memoria: se realizaron consultas sobre el manejo de matrices bidimensionales, el acceso mediante índices, la representación de los datos en pantalla y las posibles situaciones de error durante la modificación de registros.

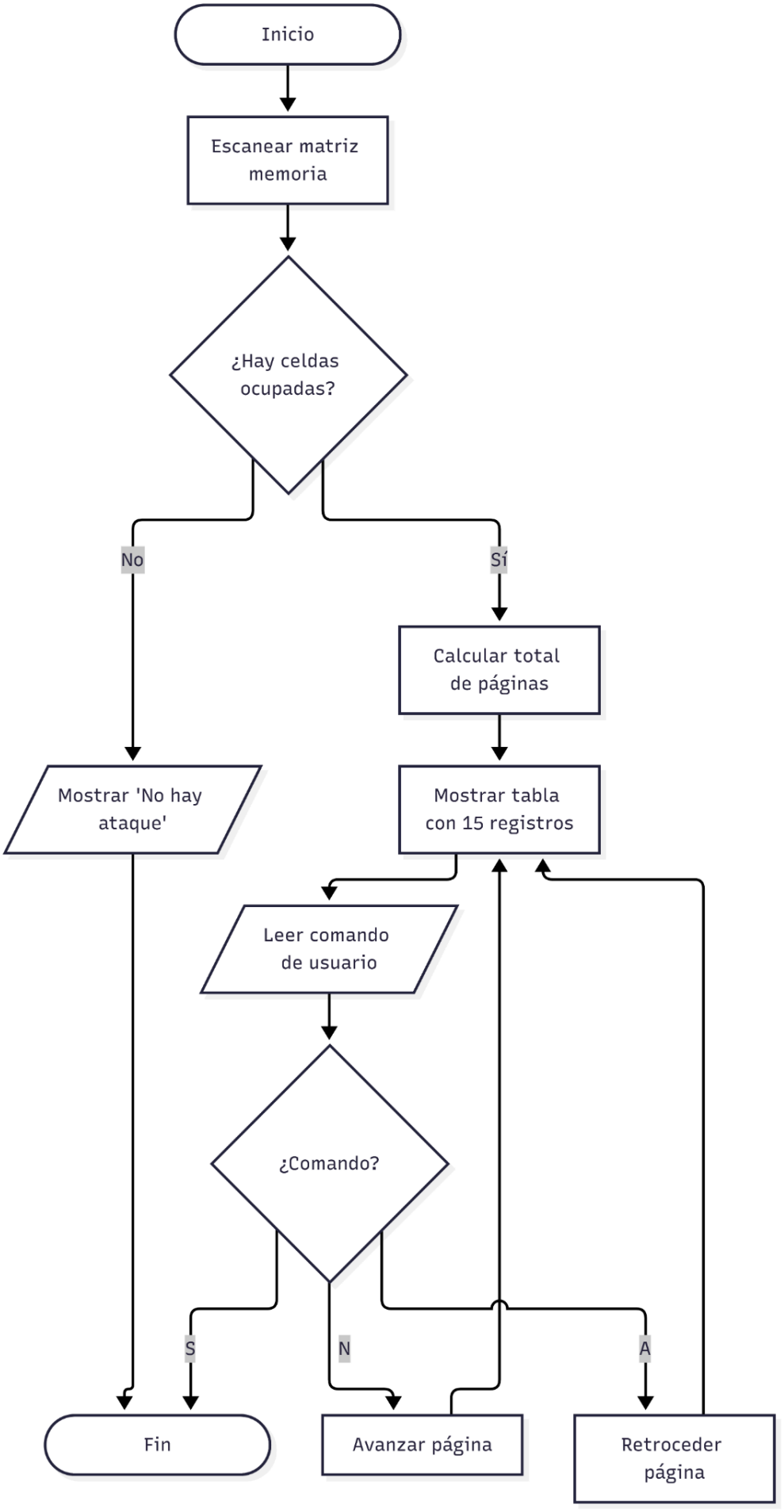
- Compilación modular: se solicitó orientación para organizar la compilación de los distintos archivos fuente mediante un Makefile, así como para identificar y solucionar errores relacionados con los nombres y la vinculación de archivos.

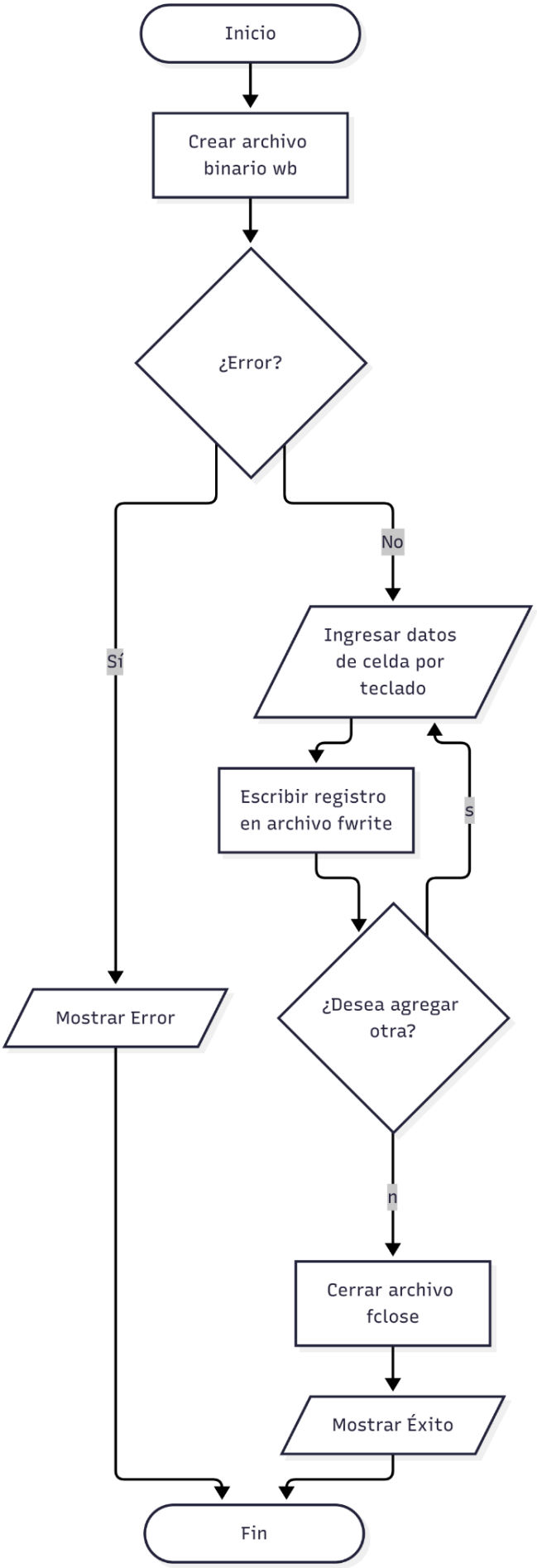
- Pruebas y documentación: se utilizó la IA como apoyo para identificar casos de prueba, contemplar situaciones excepcionales, verificar el funcionamiento de las operaciones implementadas y mejorar la organización y redacción del informe.

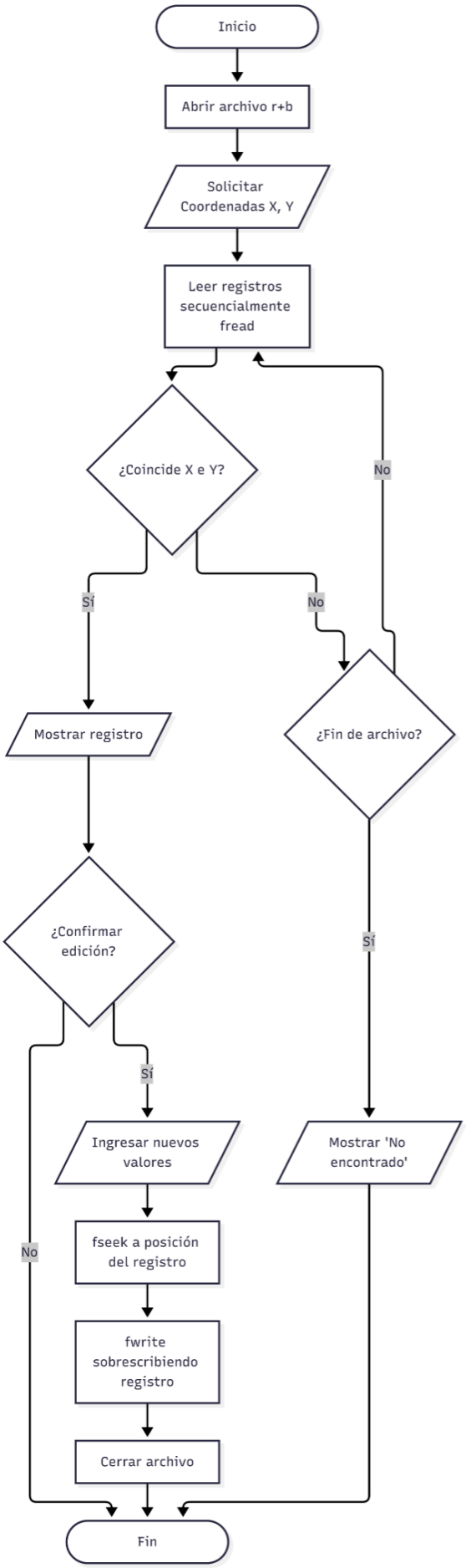
Las respuestas y sugerencias proporcionadas por la herramienta fueron consideradas como orientación y evaluadas por los integrantes del grupo. Las decisiones finales, la adaptación de las propuestas, la implementación del código y la verificación de su funcionamiento estuvieron a cargo de los integrantes, quienes conservaron la responsabilidad sobre el desarrollo del trabajo práctico.

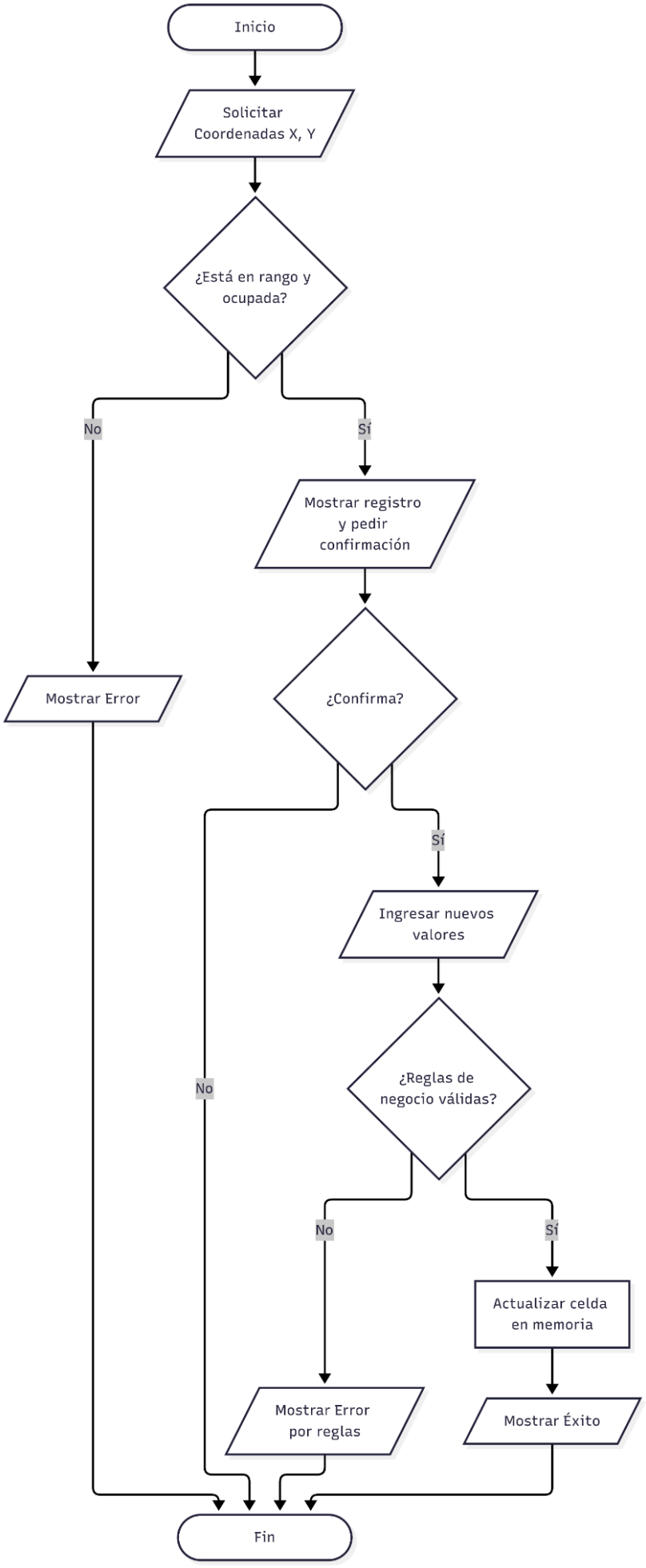
5. Anexo: Diagramas de Flujo de Subprogramas.

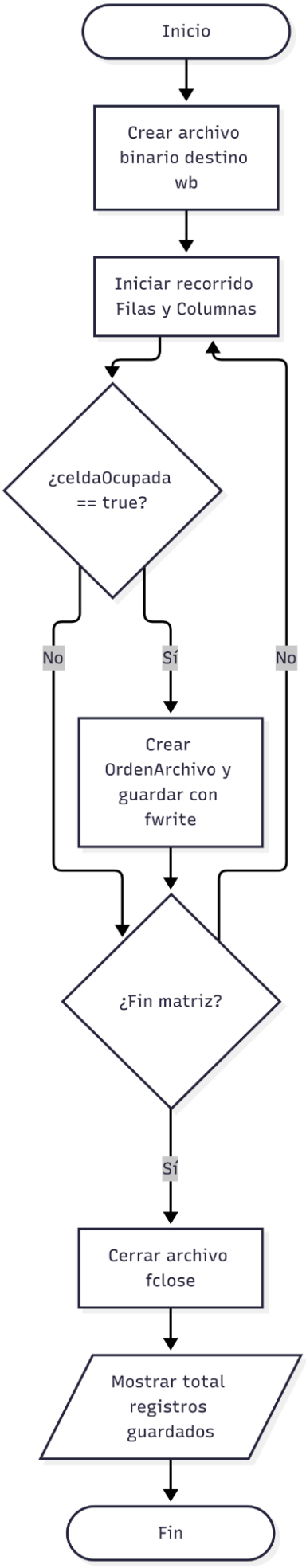


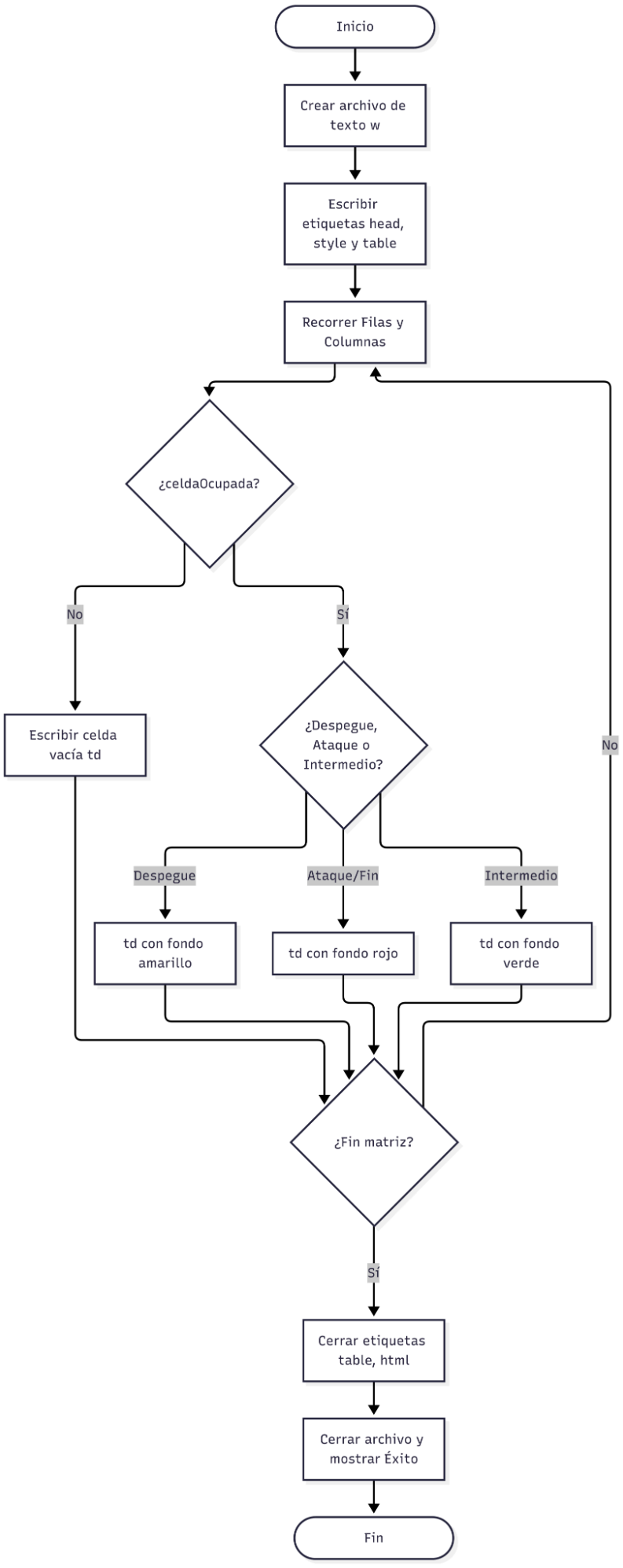

Inicio
Escanear matriz memoria
¿Hay celdas ocupadas?
No
Si
Calcular total de páginas
Mostrar 'No hay ataque'
Mostrar tabla con 15 registros
Leer comando de usuario
¿Comando?
S
N
A
Fin
Avanzar página
Retroceder página


Inicio
Crear archivo binario wb
¿Error?
Si
No
Ingresar datos de celda por teclado
Escribir registro en archivo fwrite
s
Mostrar Error
¿Desea agregar otra?
n
Cerrar archivo fclose
Mostrar Éxito
Fin


Inicio
Abrir archivo r+b
Solicitar Coordenadas X, Y
Leer registros secuencialmente fread
¿Coincide X e Y?
No
Sí
No
Mostrar registro
¿Fin de archivo?
¿Confirmar edición?
Sí
Sí
Ingresar nuevos valores
Mostrar 'No encontrado'
No
fseek a posición del registro
fwrite sobrescribiendo registro
Cerrar archivo
Fin


Inicio
Solicitar Coordenadas X, Y
¿Está en rango y ocupada?
No
Sí
Mostrar Error
Mostrar registro y pedir confirmación
¿Confirma?
No
Sí
Ingresar nuevos valores
¿Reglas de negocio válidas?
No
Sí
Mostrar Error por reglas
Actualizar celda en memoria
Mostrar Éxito
Fin


Inicio
Crear archivo binario destino wb
Iniciar recorrido Filas y Columnas
¿celdaOcupada == true?
No
Sí
No
Crear OrdenArchivo y guardar con fwrite
¿Fin matriz?
Sí
Cerrar archivo fclose
Mostrar total registros guardados
Fin


Inicio
Crear archivo de texto w
Escribir etiquetas head, style y table
Recorrer Filas y Columnas
¿celdaOcupada?
No
Sí
Escribir celda vacía td
¿Despegue, Ataque o Intermedio?
No
Despegue
Ataque/Fin
Intermedio
td con fondo amarillo
td con fondo rojo
td con fondo verde
¿Fin matriz?
Sí
Cerrar etiquetas table, html
Cerrar archivo y mostrar Éxito
Fin